

End-to-End E-commerce Intelligence System: Building a Customer Analytics Framework 360

Step 1: Data Loading and Initial Exploration

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from commonUtil import readDataset, identify_primary_key_column,
handle_missing_values, remove_duplicated, verify_date_column,
standarize_columns, validate_customers_column

#### Load each dataset
####Inspect structure using .head(), .info(), .describe() step-1
customers = readDataset("dataset/customers.csv") #loading customer
table
category_translation = readDataset("dataset/category_translation.csv")
#loading category_translation table
location = readDataset("dataset/location.csv") #loading customer table
order_item = readDataset("dataset/order_item.csv") #loading location
table
orders = readDataset("dataset/orders.csv") #loading orders table
payments = readDataset("dataset/payments.csv") #loading payments table
products = readDataset("dataset/products.csv") #loading products table
reviews = readDataset("dataset/reviews.csv") #loading reviews table
sellers = readDataset("dataset/sellers.csv") #loading sellers table

### Identify primary
print("\n \n \n#### Identifying Primary key for each dataset ###")
# identify_primary_key_column(customers)# customers dataset primary
key identification
# identify_primary_key_column(category_translation)#
category_translation dataset primary key identification
# identify_primary_key_column(location)# location dataset primary key
identification
# identify_primary_key_column(order_item)# order_item dataset primary
key identification
# identify_primary_key_column(orders)# orders dataset primary key
identification
# identify_primary_key_column(payments) # payments dataset primary key
identification
# identify_primary_key_column(products)# products dataset
primary key identification
# identify_primary_key_column(reviews)# reviews dataset primary key
identification
colName, msg = identify_primary_key_column(sellers)# sellers dataset
primary key identification
print(colName, msg)
```

```

# #Step 2: Data Cleaning and Preprocessing
# #Handle missing values appropriately
print("\n\n\n#### Handle missing values appropriately ####")
# handle_missing_values(customers, "customers") #Handle missing values
customers dataset
# handle_missing_values(category_translation, "category_translation")
#Handle missing values category_translation dataset
# handle_missing_values(location, "location") #Handle missing values
location dataset
# handle_missing_values(order_item, "order_item") #Handle missing
values order_item dataset
# handle_missing_values(orders, "orders") #Handle missing values
orders dataset
# handle_missing_values(payments, "payments") #Handle missing values
payments dataset
# handle_missing_values(products, "products") #Handle missing values
products dataset
# handle_missing_values(reviews,"reviews") #Handle missing values
reviews dataset
tblName, message = handle_missing_values(sellers, "sellers") #Handle
missing values sellers dataset
print(tblName, message)

# #Remove duplicated values
# ### Output
print("\n\n\n#### Remove duplicated values ####")

# remove_duplidated(customers, "customers") #Remove duplicated values
customers dataset
# remove_duplidated(category_translation, "category_translation")
#Remove duplicated values category_translation dataset
# remove_duplidated(location, "location") #Remove duplicated values
location dataset
# remove_duplidated(order_item, "order_item") #Remove duplicated
values order_item dataset
# remove_duplidated(orders, "orders") #Remove duplicated values orders
dataset
# remove_duplidated(payments, "payments") #Remove duplicated values
payments dataset
# remove_duplidated(products, "products") #Remove duplicated values
products dataset
# remove_duplidated(reviews,"reviews") #Remove duplicated values
reviews dataset
#newTable, message = remove_duplidated(sellers, "sellers") #Remove
duplicated values sellers dataset
#print("##table", newTable, message)

#Convert date columns to datetime format
### Output

```

```

print("\n \n \n#### Convert date columns to datetime format ###")

# verify_date_column(customers, "customers") #Remove duplicated values
customers dataset
# verify_date_column(category_translation, "category_translation")
#Remove duplicated values category_translation dataset
# verify_date_column(location, "location") #Remove duplicated values
location dataset
# verify_date_column(order_item, "order_item") #Remove duplicated
values order_item dataset
# verify_date_column(orders, "orders") #Remove duplicated values
orders dataset
# verify_date_column(payments, "payments") #Remove duplicated values
payments dataset
# verify_date_column(products, "products") #Remove duplicated values
products dataset
# verify_date_column(reviews,"reviews") #Remove duplicated values
reviews dataset
colName1, colmsg, convertDatetime = verify_date_column(sellers,
"sellers") #Remove duplicated values sellers dataset
print(colName1, colmsg, convertDatetime)

#Validate data types and ranges
print("\n\n##Validate data types and ranges")
validate_customers_column(customers)
print("\n")

##Standardize column names if required customers dataset
print("\n\n##Standardize column names if required customers dataset")
res = standarize_columns(customers)
#tableName = standarize_columns(location, "location")
print("\n", res )

###Step 3: Data Integration (Critical Component)
##You must construct a Master Dataset by merging multiple tables.
## orders + customers
mergedTable1 = pd.merge(orders, customers, on="customer_id",
how='left')
mergedTable2 = pd.merge(mergedTable1, order_item, on="order_id",
how='left')
mergedTable3 = pd.merge(mergedTable2, products, on="product_id",
how='left')
mergedTable4 = pd.merge(mergedTable3, payments, on="order_id",
how='left')
mergedTable5 = pd.merge(mergedTable4, reviews , on="order_id",
how='left')
mergedTable6 = pd.merge(mergedTable5, sellers , on="seller_id",
how='left')
mergedTable7 = pd.merge(mergedTable6, category_translation,

```

```

on="product_category_name", how='left')
#Standardizing the columns
res = standarize_columns(mergedTable7)
#Removing duplicates
newTable = remove_dupllicated(res, "modified table") ##Final
eCommerceTable
eCommereTable = newTable
print("\n#### Final merged eCommerceTable ### \n\n\n", eCommereTable)

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99441 entries, 0 to 99440
Data columns (total 5 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customer_id                          99441 non-null  object
1   customer_unique_id                   99441 non-null  object
2   customer_zip_code_prefix             99441 non-null  int64
3   customer_city                        99441 non-null  object
4   customer_state                       99441 non-null  object

```

dtypes: int64(1), object(4)

memory usage: 3.8+ MB

```

<class 'pandas.core.frame.DataFrame'>

```

RangeIndex: 71 entries, 0 to 70

Data columns (total 2 columns):

```

#   Column                                Non-Null Count  Dtype
---  -
0   product_category_name                 71 non-null    object
1   product_category_name_english         71 non-null    object

```

dtypes: object(2)

memory usage: 1.2+ KB

```

<class 'pandas.core.frame.DataFrame'>

```

RangeIndex: 1000163 entries, 0 to 1000162

Data columns (total 5 columns):

```

#   Column                                Non-Null Count  Dtype
---  -
0   geolocation_zip_code_prefix           1000163 non-null  int64
1   geolocation_lat                       1000163 non-null  float64
2   geolocation_lng                       1000163 non-null  float64
3   geolocation_city                      1000163 non-null  object
4   geolocation_state                     1000163 non-null  object

```

dtypes: float64(2), int64(1), object(2)

memory usage: 38.2+ MB

```

<class 'pandas.core.frame.DataFrame'>

```

RangeIndex: 112650 entries, 0 to 112649

Data columns (total 7 columns):

```

#   Column                                Non-Null Count  Dtype
---  -
0   order_id                              112650 non-null  object
1   order_item_id                         112650 non-null  int64
2   product_id                           112650 non-null  object

```

```

3   seller_id          112650 non-null object
4   shipping_limit_date 112650 non-null object
5   price              112650 non-null float64
6   freight_value      112650 non-null float64
dtypes: float64(2), int64(1), object(4)
memory usage: 6.0+ MB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99441 entries, 0 to 99440
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   order_id                             99441 non-null  object
1   customer_id                          99441 non-null  object
2   order_status                         99441 non-null  object
3   order_purchase_timestamp             99441 non-null  object
4   order_approved_at                   99281 non-null  object
5   order_delivered_carrier_date         97658 non-null  object
6   order_delivered_customer_date        96476 non-null  object
7   order_estimated_delivery_date        99441 non-null  object
dtypes: object(8)
memory usage: 6.1+ MB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 103886 entries, 0 to 103885
Data columns (total 5 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   order_id                             103886 non-null object
1   payment_sequential                   103886 non-null int64
2   payment_type                         103886 non-null object
3   payment_installments                 103886 non-null int64
4   payment_value                        103886 non-null float64
dtypes: float64(1), int64(2), object(2)
memory usage: 4.0+ MB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32951 entries, 0 to 32950
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   product_id                           32951 non-null  object
1   product_category_name                 32341 non-null  object
2   product_name_lenght                  32341 non-null  float64
3   product_description_lenght           32341 non-null  float64
4   product_photos_qty                   32341 non-null  float64
5   product_weight_g                      32949 non-null  float64
6   product_length_cm                    32949 non-null  float64
7   product_height_cm                    32949 non-null  float64
8   product_width_cm                     32949 non-null  float64
dtypes: float64(7), object(2)
memory usage: 2.3+ MB

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99224 entries, 0 to 99223
Data columns (total 7 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   review_id                            99224 non-null  object
1   order_id                             99224 non-null  object
2   review_score                         99224 non-null  int64
3   review_comment_title                 11568 non-null  object
4   review_comment_message               40977 non-null  object
5   review_creation_date                 99224 non-null  object
6   review_answer_timestamp              99224 non-null  object
dtypes: int64(1), object(6)
memory usage: 5.3+ MB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3095 entries, 0 to 3094
Data columns (total 4 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   seller_id                            3095 non-null  object
1   seller_zip_code_prefix               3095 non-null  int64
2   seller_city                         3095 non-null  object
3   seller_state                        3095 non-null  object
dtypes: int64(1), object(3)
memory usage: 96.8+ KB

```

```

#### Identifying Primary key for each dataset ###
seller_id is a primary key

```

```

#### Handle missing values appropriately ###
Before handling seller_id                                0
seller_zip_code_prefix    0
seller_city                0
seller_state              0
dtype: int64
seller_id                  0
seller_zip_code_prefix    0
seller_city                0
seller_state              0
dtype: int64 After handling Nochange

```

```

#### Remove duplicated values ###

```

```
#### Convert date columns to datetime format ###
seller_id is already a date timeformat
```

```
##Validate data types and ranges
All customer_id are unique
Customer_zip_code_prefix states are not 2 letters
All customer_state states are in correct format
```

```
##Standardize column names if required customers dataset
```

	customer_id
customer_unique_id \	
0	06b8999e2fba1a1fbc88172c00ba8bc7861eff4711a542e4b93843c6dd7febb0
1	18955e83d337fd6b2def6b18a428ac77290c77bc529b7ac935b93aa66c333dc3
2	4e7b3e00288586ebd08712fdd0374a03060e732b5b29e8181a18229c7b0b2b5e
3	b2b6027bc5c5109e529d4dc6358b12c3259dac757896d24d7702b9acbbff3f3c
4	4f2d8ab171c80ec8364f7c12e35b23ad345ecd01c38d18a9036ed96c73b8d066
...	...
...	
99436	17ddf5dd5d51696bb3d7c6291687be6f1a29b476fee25c95fbafc67c5ac95cf8
99437	e7b71a9017aa05c9a7fd292d714858e8d52a67c98be1cf6a5c84435bd38d095d
99438	5e28dfe12db7fb50a4b2f691faecea5ee9f50caf99f032f0bf3c55141f019d99
99439	56b18e2166679b8a959d72dd06da27f973c2643a0a458b49f58cea58833b192e
99440	274fa6071e5e17fe303b9748641082c884732c5050c01db9b23e19ba39899398

	customer_zip_code_prefix	customer_city	customer_state
0	14409	franca	SP
1	9790	sao bernardo do campo	SP
2	1151	sao paulo	SP
3	8775	mogi das cruze	SP

4	13056	campinas	SP
...	...	...	...
99436	3937	sao paulo	SP
99437	6764	taboao da serra	SP
99438	60115	fortaleza	CE
99439	92120	canoas	RS
99440	6703	cotia	SP
[99441 rows x 5 columns]			
total data for modified table dataset: 119143			
Duplicates before removing for modified table dataset: 0			
#### Final merged eCommerceTable ###			
order_id customer_id \ 0e481f51cbdc54678b7cc49136f2d6af79ef432eb6251297304e76186b10a928d 1e481f51cbdc54678b7cc49136f2d6af79ef432eb6251297304e76186b10a928d 2e481f51cbdc54678b7cc49136f2d6af79ef432eb6251297304e76186b10a928d 353cdb2fc8bc7dce0b6741e2150273451b0830fb4747a6c6d20dea0b8c802d7ef 447770eb9100c2d0c44946d9cf07ec65d41ce2a54c0b03bf3443c3d931a367089 11913863943bddc261676b46f01ca7ac2f7bd81fca14ff2861355f6e5f14306ff977a7 11913983c1379a015df1e13d02aae0204711ab1aa71eb042121263aafbe80c1b562c9c 11914011c177c8e97725db2631073c19f07b62b331b74b18dc79bcdcf6532d51e1637c1 11914111c177c8e97725db2631073c19f07b62b331b74b18dc79bcdcf6532d51e1637c1 11914266dea50a8b16d9b4dee7af250b4be1a5edb027a75a1449115f6b43211ae02a24			
order_status order_purchase_timestamp order_approved_at \ 0delivered2017-10-02 10:56:332017-10-02 11:07:15 1delivered2017-10-02 10:56:332017-10-02 11:07:15			

2	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15
3	delivered	2018-07-24 20:41:37	2018-07-26 03:24:27
4	delivered	2018-08-08 08:38:49	2018-08-08 08:55:23
...	...	...	...
119138	delivered	2018-02-06 12:58:58	2018-02-06 13:10:37
119139	delivered	2017-08-27 14:46:43	2017-08-27 15:04:16
119140	delivered	2018-01-08 21:28:27	2018-01-08 21:36:21
119141	delivered	2018-01-08 21:28:27	2018-01-08 21:36:21
119142	delivered	2018-03-08 20:57:30	2018-03-09 11:20:28

	order_delivered_carrier_date	order_delivered_customer_date \
0	2017-10-04 19:55:00	2017-10-10 21:25:13
1	2017-10-04 19:55:00	2017-10-10 21:25:13
2	2017-10-04 19:55:00	2017-10-10 21:25:13
3	2018-07-26 14:31:00	2018-08-07 15:27:45
4	2018-08-08 13:50:00	2018-08-17 18:06:29
...	...	...
119138	2018-02-07 23:22:42	2018-02-28 17:37:56
119139	2017-08-28 20:52:26	2017-09-21 11:24:17
119140	2018-01-12 15:35:03	2018-01-25 23:32:54
119141	2018-01-12 15:35:03	2018-01-25 23:32:54
119142	2018-03-09 22:11:59	2018-03-16 13:08:30

	order_estimated_delivery_date	customer_unique_id
\		
0	2017-10-18 00:00:00	7c396fd4830fd04220f754e42b4e5bff
1	2017-10-18 00:00:00	7c396fd4830fd04220f754e42b4e5bff
2	2017-10-18 00:00:00	7c396fd4830fd04220f754e42b4e5bff
3	2018-08-13 00:00:00	af07308b275d755c9edb36a90c618231
4	2018-09-04 00:00:00	3a653a41f6f9fc3d2a113cf8398680e8

...	...	...
119138	2018-03-02 00:00:00	da62f9e57a76d978d02ab5362c509660
119139	2017-09-27 00:00:00	737520a9aad80b3fbbdad19b66b37b30
119140	2018-02-15 00:00:00	5097a5312c8b157bb7be58ae360ef43c
119141	2018-02-15 00:00:00	5097a5312c8b157bb7be58ae360ef43c
119142	2018-04-03 00:00:00	60350aa974b26ff12caad89e55993bd6

	customer_zip_code_prefix	...
review_id \		
0	3149	...

```

a54f0611adc9ed256b57ede6b6eb5114
1 3149 ...
a54f0611adc9ed256b57ede6b6eb5114
2 3149 ...
a54f0611adc9ed256b57ede6b6eb5114
3 47813 ...
8d5266042046a06655c8db133d120ba5
4 75265 ...
e73b67b67587f7644d5bd1a52deb1b01
... ..
.
119138 11722 ...
29bb71b2760d0f876dfa178a76bc4734
119139 45920 ...
371579771219f6db2d830d50805977bb
119140 28685 ...
8ab6855b9fe9b812cd03a480a25058a1
119141 28685 ...
8ab6855b9fe9b812cd03a480a25058a1
119142 83750 ...
dc9c59b4688062c25758c2be4cafc523

```

	review_score	review_comment_title \
0	4.0	NaN
1	4.0	NaN
2	4.0	NaN
3	4.0	Muito boa a loja
4	5.0	NaN
...	...	...
119138	4.0	NaN
119139	5.0	NaN
119140	2.0	NaN
119141	2.0	NaN
119142	5.0	NaN

	review_comment_message \
0	Não testei o produto ainda, mas ele veio corre...
1	Não testei o produto ainda, mas ele veio corre...
2	Não testei o produto ainda, mas ele veio corre...
3	Muito bom o produto.
4	NaN
...	...
119138	So uma peça que veio rachado mas tudo bem rs
119139	Foi entregue antes do prazo.
119140	Foi entregue somente 1. Quero saber do outro p...
119141	Foi entregue somente 1. Quero saber do outro p...
119142	NaN

review_creation_date	review_answer_timestamp	seller_zip_code_prefix \
----------------------	-------------------------	--------------------------

```

0      2017-10-11 00:00:00      2017-10-12 03:43:48
9350.0
1      2017-10-11 00:00:00      2017-10-12 03:43:48
9350.0
2      2017-10-11 00:00:00      2017-10-12 03:43:48
9350.0
3      2018-08-08 00:00:00      2018-08-08 18:37:50
31570.0
4      2018-08-18 00:00:00      2018-08-22 19:07:58
14840.0
...
...
119138 2018-03-01 00:00:00      2018-03-02 17:50:01
17602.0
119139 2017-09-22 00:00:00      2017-09-22 23:10:57
8290.0
119140 2018-01-26 00:00:00      2018-01-27 09:16:56
37175.0
119141 2018-01-26 00:00:00      2018-01-27 09:16:56
37175.0
119142 2018-03-17 00:00:00      2018-03-17 16:33:31
14407.0

```

```

      seller_city seller_state product_category_name_english
0          maua      SP      housewares
1          maua      SP      housewares
2          maua      SP      housewares
3  belo horizonte      SP      perfumery
4          guariba      SP      auto
...
119138      tupa      SP      baby
119139  sao paulo      SP      home_appliances_2
119140  ilicinea      MG      computers_accessories
119141  ilicinea      MG      computers_accessories
119142  franca      SP      health_beauty

```

```
[119143 rows x 40 columns]
```

##Step 4: Feature Engineering

```

# ###Step 4: Feature Engineering
# ###Total order value (aggregated from order_items or payments)
#Total order value (aggregated from payments)
totalOrderValue = eCommerceTable["payment_value"].sum()
print("\n #### Total order value (aggregated from payments) ### \n",
totalOrderValue)

# ### Delivery time (order purchase to delivery date
print("\n ### Delivery time (order purchase to delivery date ### \n")

```

```

actualTime =
( pd.to_datetime(eCommerceTable['order_delivered_customer_date']) -
pd.to_datetime(eCommerceTable['order_purchase_timestamp']))
print(actualTime.head(5))### Delivery time (order purchase to delivery
date ###

# ### Number of items per order
noOfItems = eCommerceTable.groupby('order_id').size()
print("\n\n ###Number of items per order \n", noOfItems.head(5))

# ### Customer purchase frequency
customerFrequency = eCommerceTable.groupby('customer_id')
['order_id'].nunique()
filteredData = eCommerceTable[eCommerceTable['order_status'] ==
"delivered"]
filteredData =
filteredData.assign(convertedDate=pd.to_datetime(filteredData['order_p
urchase_timestamp']))
filteredData = filteredData.sort_values(['customer_unique_id',
'convertedDate'])
print(filteredData)
print("\n ###Customer purchase frequency \n",
filteredData.groupby('customer_unique_id')['convertedDate'].diff())

# ### Customer lifetime value (basic approximation)
spendBycustomer = eCommerceTable.groupby('customer_id')
['payment_value'].sum()
print("\n ###Customer lifetime value (basic approximation) \n",
spendBycustomer.head())

# ###Average order value per customer
sumData = eCommerceTable['payment_value'].sum()
countData = eCommerceTable['customer_unique_id'].count()
print("\n ###Average order value per customer \n", sumData /
countData) ## Calculating the avg value by taking sum of payment_value
divided by count of customer (customer unique id)

#### Total order value (aggregated from payments) ###
20579664.009999998

### Delivery time (order purchase to delivery date ###

0      8 days 10:28:40
1      8 days 10:28:40
2      8 days 10:28:40
3     13 days 18:46:08
4      9 days 09:27:40
dtype: timedelta64[ns]

```

###Number of items per order

order_id	
00010242fe8c5a6d1ba2dd792cb16214	1
00018f77f2f0320c557190d7a144bdd3	1
000229ec398224ef6ca0657da4fc703e	1
00024acbcd0a6daa1e931b038114c75	1
00042b26cf59d7ce69dfabb4e55b4fd9	1

dtype: int64

customer_id \	order_id
63195	e22acc9c116caa3f2b7121bbb380d08e
	fadb3709178fc513abc1b2670aa1ad2
88385	3594e05a005ac4d06a72673270ef9ec9
	4cb282e167ae9234755102258dd52ee8
31704	b33ec3b699337181488304f362a6b734
	9b3932a6253894a02c1df9d19004239f
117980	41272756ecddd9a9ed0180413cc22fb6
	914991f0c02ef0843c0e7010c819d642
49813	d957021f1127559cd947b62533f484f7
	47227568b10f5f58a524a75507e6992c
...	...
...	...
104109	725cf8e9c24e679a8a5a32cb92c9ce1e
	74be082247cd677a147d83ee670e9d53
24594	c71b9252fd7b3b263aaa4cb09319a323
	0ecf8e0a08148af0bf313184f167670a
52660	fdc45e6c7555e6cb3cc0daca2557dbe1
	27f584b0f1dc4e610065c240f68b6be0
115992	94d3ee0bc2a0af9d4fa47a4d63616e8d
	832a3b0254347d409512ae92eaf154a6
83699	f79a35da168301ae56922475da21117b
	70bc4d9cf2bb3ea3f7b09798370f5884

	order_status	order_purchase_timestamp	order_approved_at \
63195	delivered	2018-05-10 10:56:27	2018-05-10 11:11:18
88385	delivered	2018-05-07 11:11:27	2018-05-07 18:25:44
31704	delivered	2017-03-10 21:05:03	2017-03-10 21:05:03
117980	delivered	2017-10-12 20:29:41	2017-10-12 20:49:17
49813	delivered	2017-11-14 19:45:42	2017-11-14 20:06:52
...	...	...	...
104109	delivered	2017-06-08 21:00:36	2017-06-08 21:15:16
24594	delivered	2017-12-10 20:07:56	2017-12-10 20:16:20
52660	delivered	2017-02-07 15:49:16	2017-02-07 16:02:44
115992	delivered	2018-05-02 15:17:41	2018-05-02 15:58:47
83699	delivered	2017-05-02 20:18:45	2017-05-03 15:15:13

	order_delivered_carrier_date	order_delivered_customer_date \
63195	2018-05-12 08:18:00	2018-05-16 20:48:37
88385	2018-05-09 12:18:00	2018-05-10 18:02:42

31704	2017-03-13 12:58:30	2017-04-05 14:38:47
117980	2017-10-13 20:08:19	2017-11-01 21:23:05
49813	2017-11-16 19:52:10	2017-11-27 23:08:56
...	...	...
104109	2017-06-23 13:03:27	2017-07-06 09:22:00
24594	2017-12-21 16:27:01	2018-01-09 22:28:20
52660	2017-02-08 10:04:37	2017-02-22 12:45:04
115992	2018-05-03 16:45:00	2018-05-14 11:54:26
83699	2017-05-04 12:05:22	2017-05-10 09:33:48

	order_estimated_delivery_date	customer_unique_id
\		
63195	2018-05-21 00:00:00	0000366f3b9a7992bf8c76cfd3221e2
88385	2018-05-15 00:00:00	0000b849f77a49e4a4ce2b2a4ca5be3f
31704	2017-04-07 00:00:00	0000f46a3911fa3c0805444483337064
117980	2017-11-13 00:00:00	0000f6ccb0745a6a4b88665a16c9f078
49813	2017-12-05 00:00:00	0004aac84e0df4da2b147fca70cf8255
...	...	...
104109	2017-08-02 00:00:00	fffcf5a5ff07b0908bd4e2dbc735a684
24594	2018-01-12 00:00:00	fffea47cd6d3cc0a88bd621562a9d061
52660	2017-03-24 00:00:00	ffff371b4d645b6ecea244b27531430a
115992	2018-05-28 00:00:00	ffff5962728ec6157033ef9805bacc48
83699	2017-05-26 00:00:00	ffffd2657e2aad2907e67c3e9daecbeb

	customer_zip_code_prefix	...	review_score
review_comment_title \			
63195	7787	...	5.0
Recomendo			Super
88385	6053	...	4.0
NaN			
31704	88115	...	3.0
NaN			
117980	66812	...	4.0
NaN			
49813	18040	...	5.0
NaN			
...	...	...	...
.			..
104109	55250	...	5.0

NaN			
24594	44054	...	4.0
NaN			
52660	78552	...	5.0
NaN			
115992	29460	...	5.0

83699	83608	...	5.0
NaN			
		review_comment_message	\
63195	Adorei a cortina, ficou linda na minha sala, e...		
88385		NaN	
31704		NaN	
117980		Bom vendedor	
49813		NaN	
...		...	
104109		NaN	
24594		NaN	
52660		NaN	
115992	excelente lojas, a posterior vou fazer uma aqu...		
83699		NaN	
	review_creation_date	review_answer_timestamp	
seller_zip_code_prefix			\
63195	2018-05-17 00:00:00	2018-05-21 13:44:48	
13405.0			
88385	2018-05-11 00:00:00	2018-05-11 22:29:25	
2215.0			
31704	2017-04-06 00:00:00	2017-04-07 01:19:44	
13232.0			
117980	2017-11-02 00:00:00	2017-11-03 00:17:51	
3569.0			
49813	2017-11-28 00:00:00	2017-12-02 22:29:35	
12327.0			
...	...	...	
...			
104109	2017-07-07 00:00:00	2017-07-11 11:20:05	
3908.0			
24594	2018-01-10 00:00:00	2018-01-15 11:51:59	
18500.0			
52660	2017-02-23 00:00:00	2017-02-25 15:03:21	
30550.0			
115992	2018-05-15 00:00:00	2018-05-17 15:16:20	
14050.0			
83699	2017-05-11 00:00:00	2017-05-13 10:55:24	
9015.0			
	seller_city	seller_state	
product_category_name_english			\

63195	piracicaba	SP
bed_bath_table		
88385	sao paulo	SP
health_beauty		
31704	campo limpo paulista	SP
stationery		
117980	sao paulo	SP
telephony		
49813	jacarei	SP
telephony		
...	...	...
..		
104109	sao paulo	SP
health_beauty		
24594	laranjal paulista	SP
baby		
52660	belo horizonte	MG
auto		
115992	ribeirao preto	SP
watches_gifts		
83699	santo andre	SP
perfumery		

	convertedDate
63195	2018-05-10 10:56:27
88385	2018-05-07 11:11:27
31704	2017-03-10 21:05:03
117980	2017-10-12 20:29:41
49813	2017-11-14 19:45:42
...	...
104109	2017-06-08 21:00:36
24594	2017-12-10 20:07:56
52660	2017-02-07 15:49:16
115992	2018-05-02 15:17:41
83699	2017-05-02 20:18:45

[115723 rows x 41 columns]

###Customer	purchase frequency
63195	NaT
88385	NaT
31704	NaT
117980	NaT
49813	NaT
...	
104109	0 days
24594	NaT
52660	NaT
115992	NaT
83699	NaT


```
Name: convertedDate, Length: 115723, dtype: timedelta64[ns]
```

```
###Customer lifetime value (basic approximation)
```

```
customer_id
```

```
00012a2ce6f8dcda20d059ce98491703    114.74
```

```
000161a058600d5901f007fab4c27140     67.41
```

```
0001fd6190edaaaf884bcaf3d49edf079    195.42
```

```
0002414f95344307404f0ace7a26f1d5    179.35
```

```
000379cdec625522490c315e70c7a9fb    107.01
```

```
Name: payment_value, dtype: float64
```

```
###Average order value per customer
```

```
172.7307857784343
```

```
### Step 5: Exploratory Data Analysis (EDA)
```

```
### Perform structured analysis across the following dimensions:
```

```
### Customer Analysis
```

```
### ### New vs repeat customers
```

```
print("\n\n ###High-value vs low-value customers ### \n")
```

```
newCoustomerCount = eCommerceTable.groupby('customer_unique_id')
```

```
['order_id'].nunique().reset_index()
```

```
if (newCoustomerCount['order_id'] > 1).any():
```

```
    print("Repeat customers exist")
```

```
if (newCoustomerCount['order_id'] > 1).sum() == 0:
```

```
    print("New customers")
```

```
### High-value vs low-value customers
```

```
print("\n\n ###High-value vs low-value customers ### \n")
```

```
highLowValue = eCommerceTable.groupby('customer_unique_id').agg({
```

```
    min_payment=('payment_value', 'min'),
```

```
    max_payment=('payment_value', 'max')
```

```
}).reset_index()
```

```
print(highLowValue)
```

```
### Geographic distribution of customers
```

```
print("\n\n ###Geographic distribution of customers ### \n")
```

```
geographicstateCustomers =
```

```
eCommerceTable.groupby('customer_state').agg({
```

```
    "customer_id": "count"
```

```
})
```

```
geographiccityCustomers =
```

```
eCommerceTable.groupby('customer_city').agg({
```

```
    "customer_id": "count"
```

```
})
```

```
# print("\n\n ###Geographic distribution of customers state wise ### \n")
```

```
# print(geographicstateCustomers)
```

```

# print("\n\n###Geographic distribution of customers city wise### \n")
# print(geographiccityCustomers)

###Monthly revenue trends
###Revenue and Order Analysis
print("\n\n ###Revenue and Order Analysis## \n")

eCommerceTable["orderconvertDate"] = pd.to_datetime(
    eCommerceTable["order_purchase_timestamp"]
)
eCommerceTable["monthbasis"] =
eCommerceTable['orderconvertDate'].dt.to_period("M")
monthly_revenue = eCommerceTable.groupby("monthbasis").agg(
    totalOrder=("order_id", "count"),
    totalRevenue=("payment_value", "sum")
).reset_index()
print(monthly_revenue)

###Order volume trends
print("\n\n ###Order volume trends monthly ## \n")
orderVolume = monthly_revenue.sort_values("totalOrder",
ascending=False).reset_index()
print(orderVolume["totalOrder"].head())
###Peak sales periods
print("\n\n ### Peak sales periods  ## \n")
print(orderVolume["totalRevenue"].head(1))

###Product Analysis
print("\n\n ### Product Analysis  ## \n")
###Top-selling product categories
print("\n\n ### Top-selling product categories  ## \n")

topSellingCategory =
eCommerceTable.groupby("product_category_name").agg(
    topSellingCateg =("order_id", "count"),
    totalRevenue=("payment_value", "sum")
).reset_index()
print(topSellingCategory['product_category_name'].head(3))

print("\n\n ### Revenue contribution by category ## \n")
totalRevenue = topSellingCategory["totalRevenue"].sum()
print((topSellingCategory["totalRevenue"]/totalRevenue)*100) # revenue
based on the category / totalRevenue *100

###Product demand distribution
## By product
productBasedDemand = eCommerceTable.groupby("product_id").agg(
    demandByProduct =("order_id", "nunique"),
).reset_index()

```

```

print("\n\n ### Product demand distribution by Category ## \n")
print(productBasedDemand.sort_values("demandByProduct", ascending=
False))
productCategBasedDemand =
eCommerceTable.groupby("product_category_name").agg(
    demandByProductCateg =("order_id", "nunique"),
).reset_index()
print("\n\n ### Product demand distribution by product ## \n")
print(productCategBasedDemand.sort_values("demandByProductCateg",
ascending= False))

## Seller Analysis
print("\n\n ### Seller Analysis ## \n")
print("\n\n Top-performing sellers")
sellerData = eCommerceTable.groupby("product_id").agg(
    sellerCount =("order_id", "nunique"),
).reset_index()
print(sellerData.sort_values("sellerCount", ascending= False))

print("\n\n ### Seller contribution to revenue ## \n")
print("\n\n Seller contribution to revenue ")
sellerData = eCommerceTable.groupby("seller_id").agg(
    sellerContribution =("payment_value", "sum"),
).reset_index()
print(sellerData.sort_values("sellerContribution", ascending= False))

print("\n\n ### Seller distribution ## \n")
print("\n\n Seller distribution")
sellerContriData = eCommerceTable.groupby("seller_city").agg(
    sellerContribution =("payment_value", "sum"),
).reset_index()
print(sellerContriData.sort_values("sellerContribution", ascending=
False))

sellerContriData1 = eCommerceTable.groupby("seller_state").agg(
    sellerContribution =("payment_value", "sum"),
).reset_index()
print(sellerContriData1.sort_values("sellerContribution", ascending=
False))

```

###High-value vs low-value customers ###

Repeat customers exist

###High-value vs low-value customers ###

customer_unique_id	min_payment	max_payment
--------------------	-------------	-------------

0	0000366f3b9a7992bf8c76cfd3221e2	141.90	141.90
1	0000b849f77a49e4a4ce2b2a4ca5be3f	27.19	27.19
2	0000f46a3911fa3c0805444483337064	86.22	86.22
3	0000f6ccb0745a6a4b88665a16c9f078	43.62	43.62
4	0004aac84e0df4da2b147fca70cf8255	196.89	196.89
...	...	...	...
96091	fffcf5a5ff07b0908bd4e2dbc735a684	2067.42	2067.42
96092	fffea47cd6d3cc0a88bd621562a9d061	84.58	84.58
96093	ffff371b4d645b6ecea244b27531430a	112.46	112.46
96094	ffff5962728ec6157033ef9805bacc48	133.69	133.69
96095	ffffd2657e2aad2907e67c3e9daecbeb	71.56	71.56

[96096 rows x 3 columns]

###Geographic distribution of customers ###

###Revenue and Order Analysis##

	monthbasis	totalOrder	totalRevenue
0	2016-09	7	388.47
1	2016-10	401	76559.05
2	2016-12	1	19.62
3	2017-01	1035	190806.27
4	2017-02	2123	351848.13
5	2017-03	3243	547769.84
6	2017-04	2878	512126.52
7	2017-05	4485	737425.31
8	2017-06	3853	613777.41
9	2017-07	4949	749242.84
10	2017-08	5278	884168.36
11	2017-09	5186	1030141.97
12	2017-10	5683	1046083.41
13	2017-11	9191	1610581.21
14	2017-12	6645	1060949.63
15	2018-01	8655	1425461.40
16	2018-02	8057	1327338.97
17	2018-03	8615	1486669.64
18	2018-04	8277	1500373.12
19	2018-05	8258	1512010.75
20	2018-06	7403	1299683.17
21	2018-07	7376	1362267.08
22	2018-08	7524	1248942.63
23	2018-09	16	4439.54
24	2018-10	4	589.67

###Order volume trends monthly ##

```
0    9191
1    8655
2    8615
3    8277
4    8258
Name: totalOrder, dtype: int64
```

```
### Peak sales periods  ##
```

```
0    1610581.21
Name: totalRevenue, dtype: float64
```

```
### Product Analysis  ##
```

```
### Top-selling product categories  ##
```

```
0    agro_industria_e_comercio
1                alimentos
2        alimentos_bebidas
Name: product_category_name, dtype: object
```

```
### Revenue contribution by category  ##
```

```
0    0.588851
1    0.230265
2    0.136261
3    0.153711
4    0.011537
...
68   0.339670
69   0.049808
70   2.416242
71   1.026979
72   5.445092
Name: totalRevenue, Length: 73, dtype: float64
```

```
### Product demand distribution by Category  ##
```

	product_id	demandByProduct
19742	99a4788cb24856965c36a24e339b6058	467
22112	aca2eb7d00ea1a7b8ebd4e68314663af	431
8613	422879e10f46682990de24d770e7f83d	352
27039	d1c427060a0f73f6b889a5c7c61f2ac4	323
7364	389d119b48cf3043d311335e499d9c6b	311

```

...
13692 69c58bb97c8d7b75fad9e6c1d391fda6 1
13691 69c3a9ec3f1094ad11505df27ddad569 1
13690 69c2f05e34d543dc71cfe745bfac8499 1
13689 69c2cca014581e7e3756142963a69313 1
32950 fffe9eeff12fcbd74a2f2b007dde0c58 1

```

[32951 rows x 2 columns]

Product demand distribution by product

	product_category_name	demandByProductCateg
13	cama_mesa_banho	9417
11	beleza_saude	8836
32	esporte_lazer	7720
44	informatica_acessorios	6689
54	moveis_decoracao	6449
..	...	...
46	la_cuisine	13
17	cds_dvds_musicais	12
60	pc_gamer	8
37	fashion_roupa_infanto_juvenil	8
67	seguros_e_servicos	2

[73 rows x 2 columns]

Seller Analysis

Top-performing sellers

	product_id	sellerCount
19742	99a4788cb24856965c36a24e339b6058	467
22112	aca2eb7d00ea1a7b8ebd4e68314663af	431
8613	422879e10f46682990de24d770e7f83d	352
27039	d1c427060a0f73f6b889a5c7c61f2ac4	323
7364	389d119b48cf3043d311335e499d9c6b	311
...	...	...
13692	69c58bb97c8d7b75fad9e6c1d391fda6	1
13691	69c3a9ec3f1094ad11505df27ddad569	1
13690	69c2f05e34d543dc71cfe745bfac8499	1
13689	69c2cca014581e7e3756142963a69313	1
32950	fffe9eeff12fcbd74a2f2b007dde0c58	1

[32951 rows x 2 columns]

Seller contribution to revenue

Seller contribution to revenue

	seller_id	sellerContribution
1535	7c67e1448b00f6e969d365cea6b010ab	512645.19
192	1025f0e2d44d7041d6cf58b6550e0bfa	312456.49
881	4a3ca9315b744ce9f8e9374361493884	306138.80
368	1f50f920176fa81dab994f9023523100	291918.98
1013	53243585a1d6dc2643021fd1853d8905	284903.08
...	...	...
2087	ad14615bdd492b01b0d97922e87cb87f	19.21
1370	702835e4b785b67a084280efca355756	18.56
869	4965a7002cca77301c82d3f91b82e1a9	16.36
1465	77128dec4bec4878c37ab7d6169d6f26	15.22
2519	cf6f6bc4df3999b9c6440f124fb2f687	12.22

[3095 rows x 2 columns]

Seller distribution

Seller distribution

	seller_city	sellerContribution
517	sao paulo	4269639.08
227	ibitinga	1076538.05
159	curitiba	660358.07
258	itaguaquecetuba	579187.43
220	guarulhos	511826.28
..	...	...
439	resende	42.05
279	jarinu	38.37
124	cascavel	37.09
382	palotina	28.22
311	macatuba	25.69

[611 rows x 2 columns]

	seller_state	sellerContribution
22	SP	13448403.08
15	PR	1861222.48
8	MG	1571008.90
16	RJ	1101492.65
20	SC	888833.38
19	RS	562981.58
2	BA	368177.86
4	DF	137899.29
13	PE	124894.83
6	GO	112290.09
5	ES	78839.53

7	MA	52159.22
3	CE	25849.77
10	MT	23638.95
12	PB	20610.23
17	RN	13559.01
9	MS	10826.04
18	RO	5553.27
14	PI	3184.50
21	SE	2466.13
11	PA	1393.11
1	AM	1258.80
0	AC	299.84

##Step 6: Data Visualization

##Time series plots (sales trends)

```
print("\n\n Time series plots")
eCommerceTable['order_purchase_timestamp'] = pd.to_datetime(
    eCommerceTable['order_purchase_timestamp'], errors='coerce'
)
eCommerceTable['order_date'] =
eCommerceTable['order_purchase_timestamp'].dt.date
eCommerceTable['total_revenue'] = eCommerceTable['price'] +
eCommerceTable['freight_value']
daily_revenue = eCommerceTable.groupby('order_date')
['total_revenue'].sum().reset_index()
print(daily_revenue)
###plot chart
plt.figure(figsize=(12,6))
sns.lineplot(data=daily_revenue, x='order_date', y='total_revenue')
plt.title('Daily Revenue Trend')
plt.xlabel('Date')
plt.ylabel('Total Revenue')
plt.xticks(rotation=45)
plt.show()
```

####Bar charts (category performance)

```
print("\n\n Bar charts (category performance)")

categPerformance =
eCommerceTable.groupby("product_category_name").agg(
    topSellingCateg=("order_id", "count"),
    totalRevenue=("payment_value", "sum"),
    avgPrice=('price', 'mean')
).reset_index()
categPerformance['totalRevenue'] =
pd.to_numeric(categPerformance['totalRevenue'], errors='coerce')
categPerformance = categPerformance.dropna(subset=['totalRevenue',
'product_category_name'])
plt.figure(figsize=(12,6))
```



```

sns.barplot(
    data=categPerformance.head(30),
    x='totalRevenue',
    y='product_category_name',
    palette='viridis',
    hue='product_category_name',
)

plt.title('Category Performance (Revenue)')
plt.xlabel('Total Revenue')
plt.ylabel('Product Category')
plt.show()

###Histograms (distribution analysis)
print("\n\n Histograms (distribution analysis)")
###Price based histogram
sns.histplot(
    data=eCommerceTable,
    x='price',
    bins=50,
    kde=True
)

plt.title('Distribution of Product Price')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.show()

###freight_value based histogram
sns.histplot(
    data=eCommerceTable,
    x='freight_value',
    bins=50,
    kde=True
)

plt.title('Distribution of Product Price')
plt.xlabel('freight_value')
plt.ylabel('Frequency')
plt.show()

###Box plots (outlier detection)
print("\n\n Box plots (outlier detection) ")
###Price based histogram
sns.boxplot(
    data=eCommerceTable,
    x='price',
    color="blue"
)

```

```

plt.title('Outlier detection')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.show()

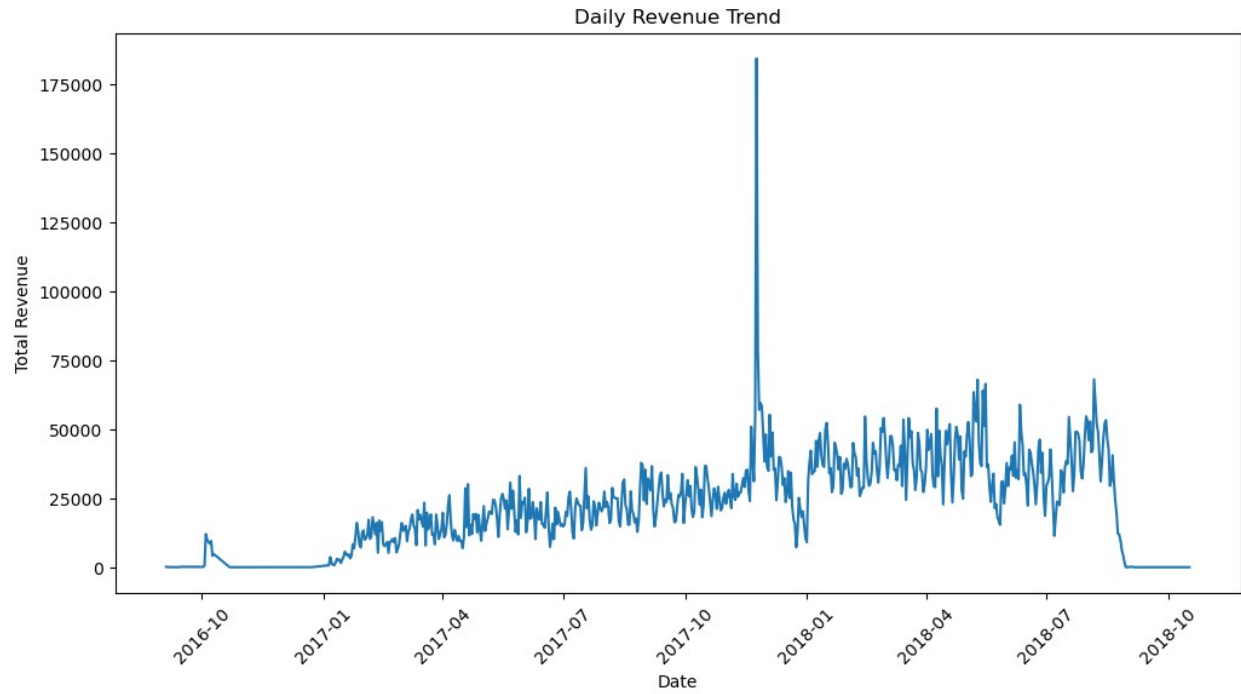
##Heatmaps (correlation analysis)
print("\n\n Heatmaps (correlation analysis)")
numeric_data = eCommerceTable[['price', 'freight_value']]
corr_matrix = numeric_data.corr()
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm',
    vmin=-1, vmax=1
)
plt.title('Heatmaps')
plt.show()

```

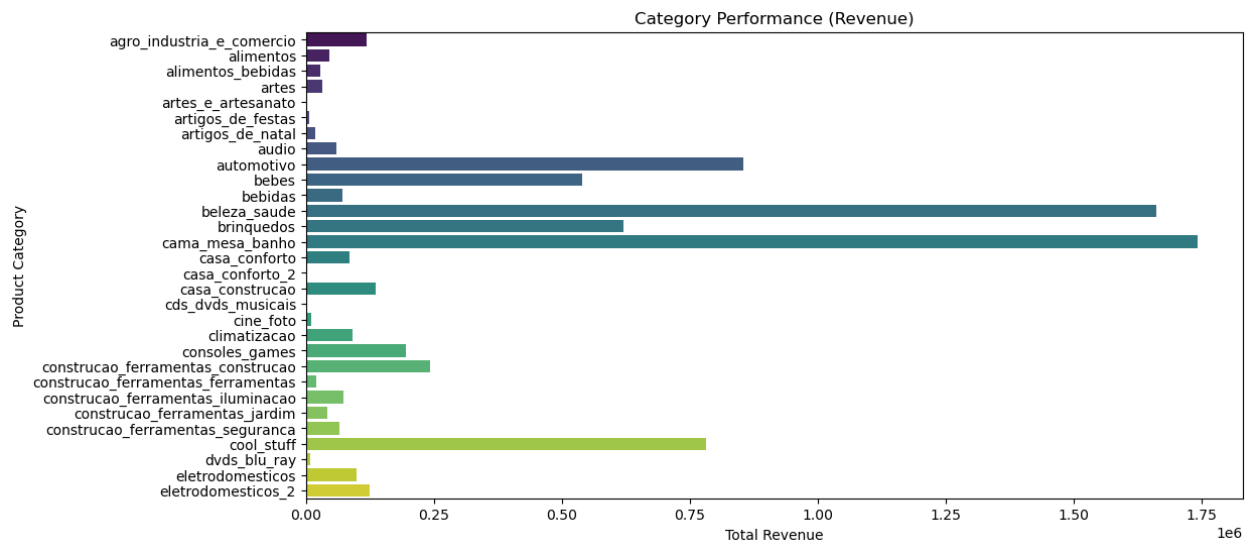
Time series plots

	order_date	total_revenue
0	2016-09-04	136.23
1	2016-09-05	75.06
2	2016-09-13	0.00
3	2016-09-15	143.46
4	2016-10-02	109.34
...	...	...
629	2018-09-29	0.00
630	2018-10-01	0.00
631	2018-10-03	0.00
632	2018-10-16	0.00
633	2018-10-17	0.00

[634 rows x 2 columns]

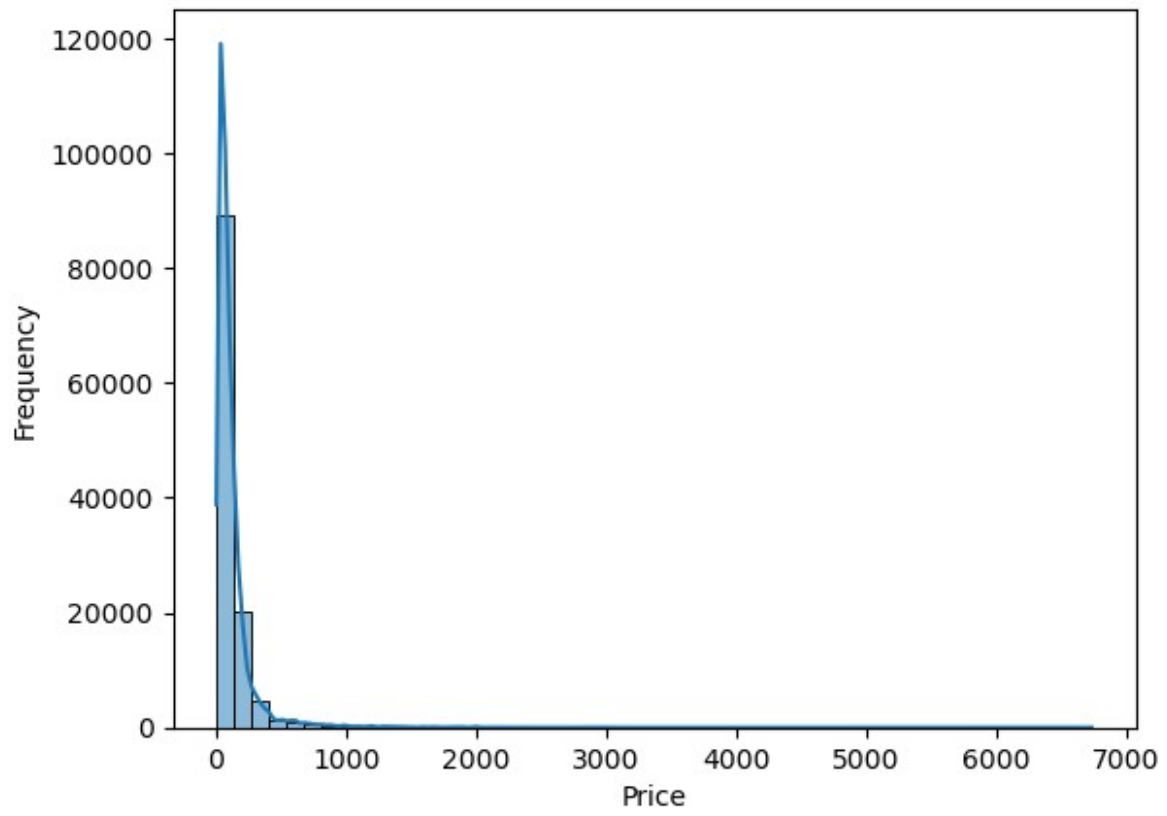


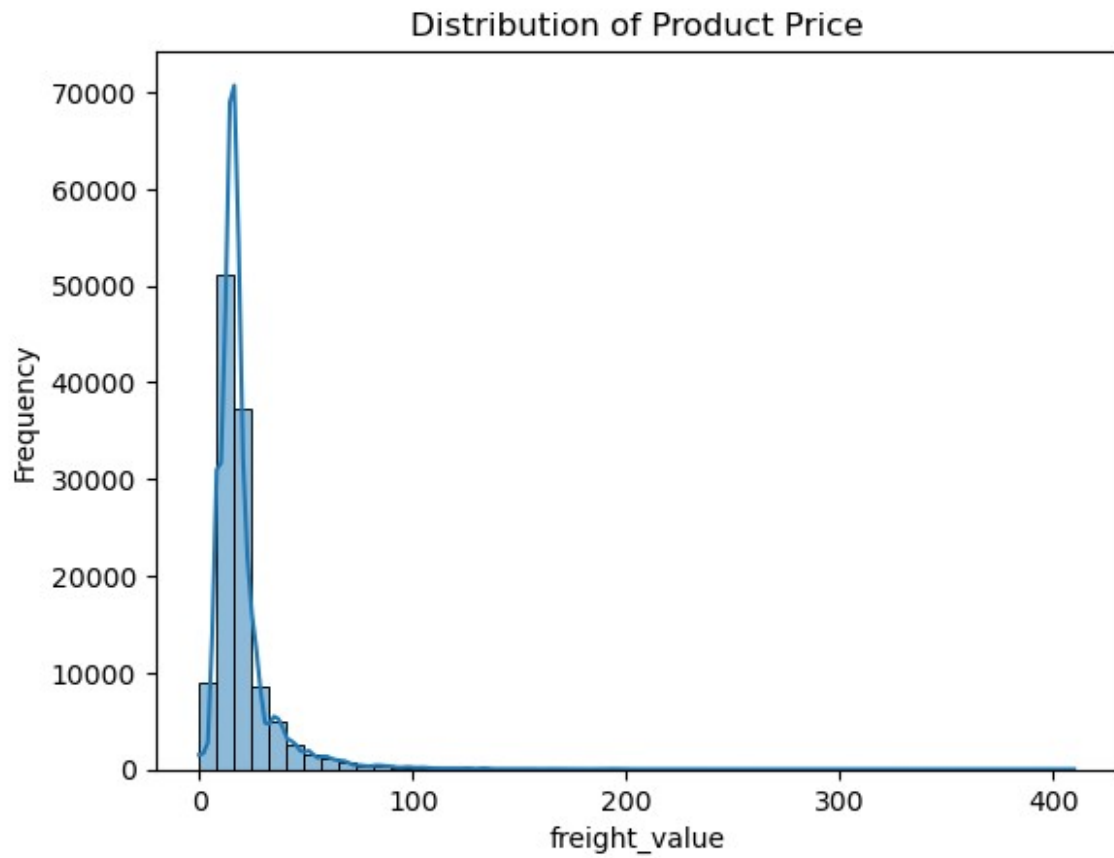
Bar charts (category performance)



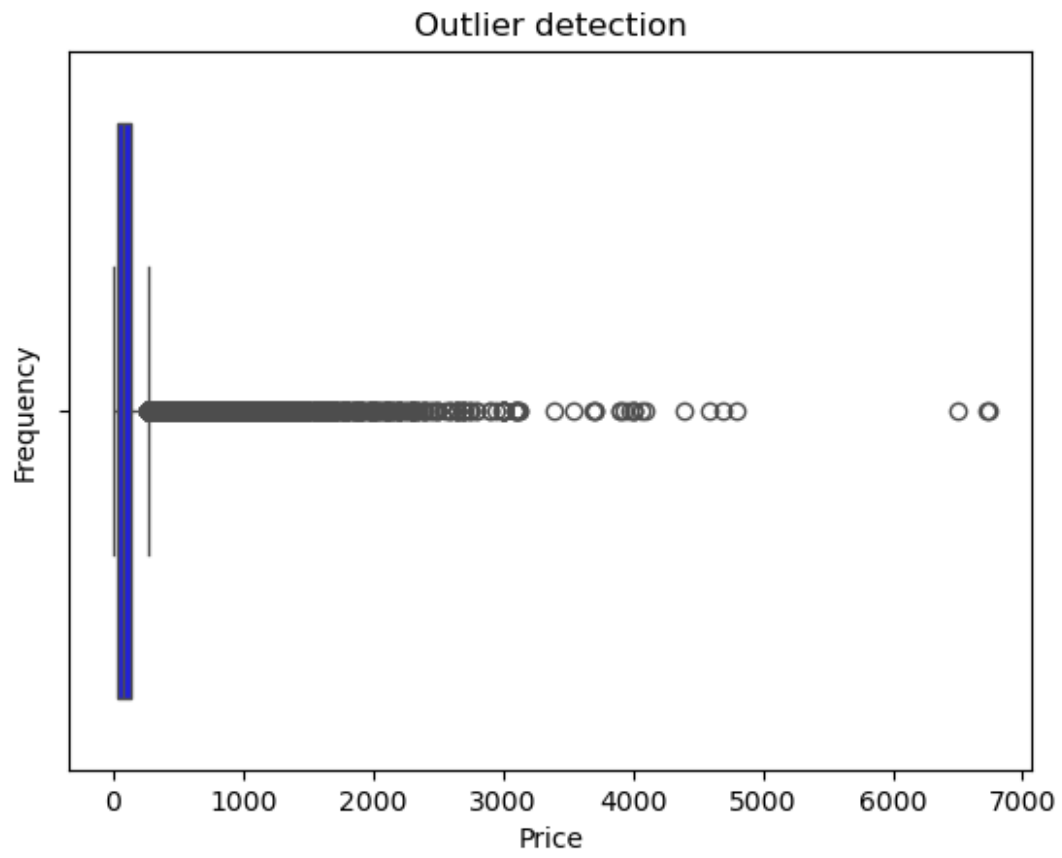
Histograms (distribution analysis)

Distribution of Product Price





Box plots (outlier detection)



Heatmaps (correlation analysis)

